Users and Access Rights

Objective

This module is to have an understanding of different user types in UNIX and their access rights.

Course Content

- User types
- File access permissions
- Changing file access permissions using symbolic mode
- Changing file access permissions using octal mode

User Types in Unix

There are three types of accounts in a Unix system:
- Root Account
- System Account
- User Account

Root	System	User
• This is also called superuser and would have complete and unfettered control of the system. • A superuser can run any commands without any restriction. This user should be assumed as a system administrator.	• System accounts are those needed for the operation of system-specific components for example mail accounts and the sshd accounts. • These accounts are usually needed for some specific function on your system, and any modifications to them could adversely affect the system.	• User accounts provide interactive access to the system for users and groups of users. • General users are typically assigned to these accounts and usually have limited access to critical system files and directories.

Managing users and groups

Unix supports a concept of Group which logically groups a number of accounts.
➢ Every account would be a part of any group.
➢ Unix groups plays important role in handling file permissions and process management.
➢ Grouping of users allows to grant and revoke file permissions collectively.

File Permission & Ownership level
File ownership is an important component of Unix that provides a secure method for storing files. Every file in Unix has the following attributes:

Permission	Symbol	Description
Owner Permissions	u	The owner's permissions determine what actions the owner of the file can perform on the file.
Group Permissions	g	The group's permissions determine what actions a user, who is a member of the group that a file belongs to, can perform on the file.
Other Permissions	o	The permissions for others indicate what action all other users can perform on the file.

File Permission Types (Mode)
Three type of permissions can be set for any ownership level:

- Read
- Write
- Execute

These three modes or permissions have different meaning for a file and a directory;

Mode	Symbol	For File	For Directory
Read	r	Allows to view the content	Allows to view the directory content using **ls** command
Write	w	Allows edit and save changes.	Create and delete files in the directory
Execute	x	Allows to run program or script	Allows to access the files in the directory

Changing Permissions (chmod)
To change file or directory permissions, the **chmod** (change mode) command is used. There are two ways to use chmod:

- Symbolic mode

- Absolute mode

Syntax:

```
$ chmod [OPTION] MODE          FILENAME
$ chmod [OPTION] OCTAL-MODE FILENAME
```

chmod command options:

-f, --silent, --quiet suppress most error messages

-R, --recursive change files and directories recursively

--help display this help and exit

Using chmod in Symbolic Mode:

With symbolic representation, permission set can be added, deleted, or specified using the operators listed in the following table:

chmod operators	Description
+	Adds the designated permission(s) to a file or directory.
-	Removes the designated permission(s) from a file or directory.
=	Sets the designated permission(s).

Example:

Let us assume there is a file name "abc" , whose initial permissions are only read permission to user and group and others. We will provide full permissions to all three types of users.

Let us verify the existing file permissions using ls -l command as below and observe the next steps, like changing the permissions and verifying the changes.

```
-bash-3.2$ ls -l abc
-r--r--rwx 1 735873 oinstall 0 Feb  7 12:37 abc
-bash-3.2$ chmod u+wx,g+wx abc
-bash-3.2$ ls -l abc
-rwxrwxrwx 1 735873 oinstall 0 Feb  7 12:37 abc
-bash-3.2$ chmod g-x abc
-bash-3.2$ ls -l abc
-rwxrw-rwx 1 735873 oinstall 0 Feb  7 12:37 abc
-bash-3.2$ chmod o=r-- abc
-bash-3.2$ ls -l abc
-rwxrw-r-- 1 735873 oinstall 0 Feb  7 12:37 abc
-bash-3.2$ chmod a+rwx abc
-bash-3.2$ ls -l abc
-rwxrwxrwx 1 735873 oinstall 0 Feb  7 12:37 abc
```

In the above example,

chmod u+wx,g+wx abc , adds write and execute permission to user and group to the file abc

chmod g-x abc, removes execute permission from group from the file abc.

chmod o=r-- abc, provides read permission and removes write and execute permission from others for file abc

Using chmod in Absolute(octal) Mode

The second way to modify permissions with the chmod command is to use a number to specify each set of permissions for the file. Each permission is assigned a value, as the following table shows, and the total of each set of permissions provides a number for that set.

Number	Octal Permission Representation	Ref
0	No permission	---
1	Execute permission	--x
2	Write permission	-w-
3	Execute and write permission: 1 (execute) + 2 (write) = 3	-wx
4	Read permission	r--
5	Read and execute permission: 4 (read) + 1 (execute) = 5	r-x
6	Read and write permission: 4 (read) + 2 (write) = 6	rw-
7	All permissions: 4 (read) + 2 (write) + 1 (execute) = 7	rwx

Examples: Observe the following sequence of command execution statements and analyze the output

1. Let us view the initial permissions of the file ***abc***
   ls -l ***abc***

   -rw-r--r-- 1 735873 oinstall 0 Feb 7 12:37 abc

If we observe the output, initially owner have read and write permissions and group and others have only read permission

2. Let us remove all the permission from all users for ***abc*** file, as below
   chmod 0 abc
   Let us verify the changed permissions

   ls -l abc
   ---------- 1 735873 oinstall 0 Feb 7 12:37 abc
Yes all permissions are taken away from all users on the file abc

3. To provide all permissions to the user, read permission to group and write and execute permissions for Others,

   use the below command
chmod 743 abc

Let us verify the changed permissions
ls -l abc
-rwxr---wx 1 735873 oinstall 0 Feb 7 12:37 abc
From the output all permissions to user, read permission to group and write and execute
permissions for Others, Use the below command

4. ***chmod 755 abc***
   The above command provides all permissions to the user, read and execute permissions to group and others
Let us verify ls -l abc
-rwxr-xr-x 1 735873 oinstall 0 Feb 7 12:37 abc

5. ***chmod 777 abc***
   The above command provides all permissions to all users . We can verify using below command
   ls -l abc
   -rwxrwxrwx 1 735873 oinstall 0 Feb 7 12:37 abc